🎥 Movie Analytics — Full End-to-End Project

📂 1. Importing Libraries

```
# Import essential libraries

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns
```

📘 **Purpose:** Used for loading, cleaning, analyzing, and visualizing IMDb dataset.

📥 2. Load the Dataset

```
# Load movie dataset

df = pd.read_csv("movies.csv")


# Display initial rows

print(df.head())
```

📘 **Purpose:** Reads the IMDb dataset into a DataFrame for exploration.

📄 3. Initial Data Overview

```
print(df.info())        # Data types and non-null count

print(df.describe())     # Statistical summary

print(df.shape)         # Rows & columns

print(df.columns)       # Column names
```

📘 **Purpose:** Provides a quick overview of dataset structure and contents.

🧹 4. Data Cleaning

```
# Handle missing values
df['rating'].fillna(df['rating'].mean(), inplace=True)
df['revenue'].fillna(df['revenue'].median(), inplace=True)
df.dropna(subset=['title', 'genre'], inplace=True)

# Remove duplicates
df.drop_duplicates(inplace=True)
```

📘 **Purpose:** Replaces missing data and removes duplicate entries for accuracy.

⏰ 5. Convert Date Column

```
df['release_date'] = pd.to_datetime(df['release_date'])
```

📘 **Purpose:** Converts release_date into datetime for time-series operations.

🧮 6. Create New Columns (Feature Engineering)

```
df['profit'] = df['revenue'] - df['budget']      # Profit calculation
df['profit_margin'] = (df['profit'] / df['budget']) * 100
df['release_year'] = df['release_date'].dt.year
df['decade'] = (df['release_year'] // 10) * 10
```

📘 **Purpose:** Adds meaningful metrics for business and trend analysis.

🌍 7. Basic Summary Statistics

```
print("Total Movies:", len(df))
print("Average Rating:", df['rating'].mean())
print("Average Profit Margin:", df['profit_margin'].mean())
```

📘 **Purpose:** Gives quick insight into dataset performance.

🎬 8. Top 10 Movies by Revenue

```
top_revenue = df.sort_values(by='revenue', ascending=False).head(10)
print(top_revenue[['title', 'revenue', 'rating']])
```

📘 **Purpose:** Lists the highest-earning films.

📊 9. Visualization — Top 10 Movies by Revenue

```
plt.figure(figsize=(10,5))
sns.barplot(x='title', y='revenue', data=top_revenue)
plt.xticks(rotation=45)
plt.title("Top 10 Movies by Revenue")
plt.show()
```

📘 **Purpose:** Bar chart visualization for top revenue movies.

📈 10. Average Rating by Genre

```
genre_rating = df.groupby('genre')['rating'].mean().sort_values(ascending=False)
print(genre_rating)

genre_rating.plot(kind='bar', figsize=(8,4), title="Average Rating by Genre")
plt.show()
```

📘 **Purpose:** Determines which genres have the highest viewer ratings.

📅 11. Yearly Trend of Movie Releases

```
yearly_trend = df.groupby('release_year')['title'].count()
yearly_trend.plot(figsize=(10,5), title="Number of Movies Released per Year")
plt.grid(True)
plt.show()
```

📘 **Purpose:** Shows how movie production changes over time.

💰 12. Profit vs Budget Visualization

```
plt.figure(figsize=(8,5))

sns.scatterplot(x='budget', y='profit', data=df)

plt.title("Profit vs Budget Relationship")

plt.grid(True)

plt.show()
```

📘 **Purpose:** Examines correlation between movie budget and profit.

🎯 13. Top 10 Directors by Average Rating

```
top_directors =
df.groupby('director')['rating'].mean().sort_values(ascending=False).head(10)

print(top_directors)
```

📘 **Purpose:** Finds directors with consistently high-rated movies.

🎥 14. Most Popular Genres (by Count)

```
genre_count = df['genre'].value_counts().head(10)

genre_count.plot(kind='bar', color='teal', figsize=(8,4), title="Top 10 Most Frequent
Genres")

plt.show()
```

📘 **Purpose:** Identifies dominant genres in the dataset.

⚖️ 15. Correlation Heatmap

```
corr = df[['budget', 'revenue', 'rating', 'profit']].corr()

sns.heatmap(corr, annot=True, cmap='coolwarm')

plt.title("Correlation Between Key Movie Metrics")

plt.show()
```

📘 **Purpose:** Displays relationships between numeric features.

🧮 16. High-Grossing Movies Filter (Boolean Filtering)

```
high_gross = df[df['revenue'] > 500_000_000]

print(high_gross[['title', 'revenue', 'genre']].head())
```

📘 **Purpose:** Extracts blockbuster movies based on revenue threshold.

🔍 17. Decade-Wise Profit Analysis

```
decade_profit = df.groupby('decade')['profit'].mean()

decade_profit.plot(kind='line', figsize=(10,5), title="Average Profit by Decade")

plt.show()
```

📘 **Purpose:** Shows how profitability evolved across decades.

🎭 18. Top Actors by Average Revenue

```
actor_revenue =
df.groupby('main_actor')['revenue'].mean().sort_values(ascending=False).head(10)

print(actor_revenue)
```

📘 **Purpose:** Lists the most commercially successful actors.

📈 19. Rating Distribution

```
sns.histplot(df['rating'], bins=20, kde=True, color='skyblue')

plt.title("Distribution of Movie Ratings")

plt.show()
```

📘 **Purpose:** Displays overall rating spread for all movies.

🧠 20. Statistical Summary Table

```
summary = {

  "Total Movies": len(df),

  "Average Rating": round(df['rating'].mean(), 2),

  "Average Profit ($M)": round(df['profit'].mean()/1e6, 2),
```

```
    "Top Genre": df['genre'].mode()[0],
    "Top Director": df['director'].mode()[0]
}
summary_df = pd.DataFrame(summary.items(), columns=['Metric', 'Value'])
print(summary_df)
```

📘 **Purpose:** Summarizes major metrics neatly for presentation.

📤 21. Export Cleaned Data

```
df.to_csv("cleaned_movies.csv", index=False)
print("✅ Cleaned movie dataset saved successfully!")
```

📘 **Purpose:** Saves processed data for reuse or ML modeling.

🔠 22. Query Optimization

```
blockbusters = df.query('revenue > 800000000 and rating >= 8.0')
print(blockbusters[['title', 'revenue', 'rating']])
```

📘 **Purpose:** Efficient filtering using query() instead of Boolean indexing.

📊 23. Top 10 Movies by Profit Margin

```
top_profit_margin = df.sort_values(by='profit_margin', ascending=False).head(10)
plt.barh(top_profit_margin['title'], top_profit_margin['profit_margin'])
plt.title("Top 10 Movies by Profit Margin (%)")
plt.xlabel("Profit Margin (%)")
plt.show()
```

📘 **Purpose:** Highlights the most financially efficient productions.

🎞️ 24. Word Cloud of Movie Genres (Optional)

```
from wordcloud import WordCloud
```

```
wordcloud = WordCloud(width=800, height=400, background_color='black').generate(' '.join(df['genre']))

plt.imshow(wordcloud, interpolation='bilinear')

plt.axis('off')

plt.title("Genre Popularity Word Cloud")

plt.show()
```

📘 **Purpose:** Visual representation of genre frequency.

⚙️ 25. Save Insights Report

```
report = {
    "Top 10 Revenue Movies": list(top_revenue['title']),
    "Top 10 Directors": list(top_directors.index),
    "Top Genres": list(genre_count.index)
}

pd.DataFrame(dict([(k, pd.Series(v)) for k, v in report.items()])).to_csv("movie_report.csv", index=False)

print("📄 Report generated successfully!")
```

📘 **Purpose:** Stores major insights into a report file for presentation.